

XFaaS: Hyperscale and Low Cost Serverless Functions at Meta

Question-Guided Bilingual Academic Reading Package

Source PDF: XFaaS.pdf

Generated on: 2026-06-26

Confirmed Reading Questions

Q1. What tradeoffs should be considered when splitting an application into separate serverless functions?

Expected answer type: tradeoff analysis

Q2. In what cases is deploying an application in a serverless fashion a poor fit, and in what cases is it a good fit? Expected answer type: comparison

Q3. What characteristics would make an application ideal for serverless deployment? Expected answer type: concept explanation

Question-Answer Review

Question-Answer Review

Source PDF: `XFaaS.pdf`

Citation format: `[p. X, Section Y, Block P-0XX]`.

Q1. What tradeoffs should be considered when splitting an application into separate serverless functions?

Relevance judgment

Relevant.

Short answer

把应用拆成多个 serverless functions

时，核心取舍不是“函数越小越好”，而是在开发便利性、隔离和弹性之外，同时评估冷启动/初始化开销、峰值容量浪费、跨区域调度把资源供给责任从用户转移给平台，若函数调用间歇且延迟要求严格，平台可能承担高额空闲和过度配置成本；若函数可延迟、可排队、SLO 和关键性，平台就能用时间转移和全局调度提高利用率。 [p. 1, Section 1 Introduction, Block P-002] [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-004] [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-005] [p. 2, Section 1.2 Solutions for Extra Hardware Costs, Block P-008]

Evidence table

Claim	Citation	Evidence from paper	How it supports the answer	拆分为 FaaS
会把过度配置成本从用户侧转移到平台侧。	[p. 1, Section 1 Introduction, Block P-002]	The ease of use of FaaS may come at the expense of extra hardware costs for cloud providers. Before FaaS, users carried the cost of underutilized VMs; with FaaS, cloud providers bear the cost of over-provisioned resources.	说明拆分决策需要考虑资源责任和成本归属，而不只是开发便利性。	
冷启动和空闲等待之间存在直接取舍。	[p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-004]	Lengthy cold-start time is a significant FaaS challenge because initialization and shutdown steps are overheads borne by the provider, while only invocation does chargeable work. If the idle wait is too long, containers waste r...		
函数越细、调用越间歇，初始化和保温策略越影响延迟与资源浪费。				
高峰值波动会导致按峰值配置浪费，配置不足又会过载。	[p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-005]	High load variance creates a tradeoff between extra hardware cost from over-provisioning and overload during surges. XFaaS sees a peak-to-trough ratio as high as 4.3, so provisioning for peak demand would create severe waste.		
说明拆分后触发模式和负载形态会影响平台容量规划。		函数可能把压力转移给下游服务，静态并发限制不足。		
	[p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-006]	FaaS functions can overload downstream services. Meta experienced outages when spikes from non-user-facing functions overloaded a social-graph database. Static per-function concurrency limits are insufficient because limits tha...	说明函数边界之外的依赖也必须纳入拆分取舍。	
		通用 worker		
受内存限制，必须用局部性约束函数分布。	[p. 9, Section 4.5.2 Locality Groups, Block P-026]	A true universal worker is infeasible because keeping every function's JIT code in every worker's memory would exceed memory capacity, and memory-hungry functions may also exhaust memory. XFaaS partitions functions and workers...	说明函数数量、代码/JIT	
缓存和内存局部性会影响平台效率。		预留配额与机会式配额体现延迟保证和成本之间的取舍。		
	[p. 10, Section 4.6 Quotas and Utilization Control, Block P-027]	XFaaS supports reserved quota and opportunistic quota. Reserved quota provides guaranteed capacity; opportunistic quota can be delayed until off-peak hours. This creates a direct tradeoff between latency guarantees and lower ha...	说明每个函数应根据业务时限选择容量模式。	

Detailed answer

首先，要评估冷启动和保温成本。论文把函数生命周期拆成初始化、调用和关闭阶段，并指出只有调用阶段是真正可计费工作，初始化 [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-004]

其次，要评估负载是否尖峰化以及是否能被推迟。XFaaS 的接收调用峰谷比可达 4.3；若按峰值配置，会严重浪费。论文的解决方案是让函数携带截止时间、关键性、配额和未来开始时间，并在容量不足时推迟可延迟 [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-005] [p. 2, Section 1.2 Solutions for Extra Hardware Costs, Block P-008] [p. 4, Section 2.4 Terminology, Block P-014]

第三，要评估下游依赖。即使 FaaS 平台自身能处理负载，函数也可能压垮数据库、缓存等下游服务；因此需要并发限制、背压和自适应速率控制，而不是只按函数本身配置 [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-006] [p. 3, Section 1.2 Solutions for Extra Hardware Costs, Block P-009] [p. 10, Section 4.7 Adaptive Concurrency Control, Block P-028]

第四, 要评估函数粒度对运行时共享、JIT 缓存和内存的影响。XFaaS 想近似每个 worker 可执行任意函数, 但真正的通用 worker 会受到 JIT 代码和函数内存需求限制, 因此需要把函数和 worker 划分为局部性组。[p. 2, Section 1.2 Solutions for Extra Hardware Costs, Block P-007] [p. 9, Section 4.5.2 Locality Groups, Block P-026]

Blocks to reread

- [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-004]■- [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-005]■- [p. 2, Section 1.1 Risk of Extra Hardware Costs, Block P-006]■- [p. 9, Section 4.5.2 Locality Groups, Block P-026]

Uncertainty

The paper answers this question well from the platform/operator perspective. It gives less direct guidance on application-level software design boundaries beyond the implications of trigger type, latency tolerance, resource usage, and downstream dependencies.

Q2. In what cases is deploying an application in a serverless fashion a poor fit, and in what cases is it a good fit?

Relevance judgment

Relevant.

Short answer

根据本文, 要求亚秒级响应、直接处于用户交互关键路径、必须严格容量保障且已有成熟自动部署能力的场景, 不适合用 serverless functions 作为主要执行方式; Meta 的信息流展示、搜索排序、视频播放属于这类例子。更适合 serverless 的场景是异步、事件驱动、队列/日志/存储/定时器/编排触发、可设置完成时限、可接受秒级到 24 小时延迟、可通过排队和机会式配额平滑到低峰执行的工作负载, 例如推荐更新、日志处理、生产力自动化、通知活动和数据转换。[p. 5, Section 3.2 Workload Examples, Block P-016] [p. 5, Section 3.2 Workload Examples, Block P-017] [p. 5, Section 3.2 Workload Examples, Block P-018] [p. 3, Section 2.2 Spiky Load, Block P-012]

Evidence table

Claim	Citation	Evidence from paper	How it supports the answer
亚秒级用户交互请求是 serverless 的较差适配场景。	[p. 5, Section 3.2 Workload Examples, Block P-016]	XFaaS workloads seldom handle user-facing interactive requests demanding sub-second response times, such as newsfeed display, search-result ranking, or video playback. At Meta, those requests are traditionally handled by long-r...	论文直接列出 Meta 很少用 XFaaS 处理这类请求。
当必须精细容量规划且部署自动化已存在时, serverless 的两个常见优势会减弱。	[p. 5, Section 3.2 Workload Examples, Block P-017]	The paper identifies two common benefits of serverless functions: pay-as-you-go with no upfront capacity planning, and streamlined deployment where developers write code while the platform handles deployment. For user-facing re...	解释为什么 Meta 的强用户体验场景不依赖 FaaS。
异步推荐、日志处理、自动化、通知和数据转换是更适合的例子。	[p. 5, Section 3.2 Workload Examples, Block P-018]	Good XFaaS examples include asynchronous recommendations after user events, log processing that is not on the critical path but still has an SLO, productivity automation triggered by events, notification campaigns, and Morphing...	给出论文中的正面 serverless 工作负载样例。
队列、定时器、存储和编排触发通常比直接 RPC/HTTP 更能容忍延迟。	[p. 3, Section 2.2 Spiky Load, Block P-012]	XFaaS clients submit function calls in a highly spiky manner. One key insight is that some functions do not need immediate execution; functions triggered by queues, timers, storage, orchestration, and event-bridge workflows ten...	提供判断是否适合 serverless 的触发方式标准。
机会式配额可把函数推迟到低峰执行并降低峰值容量。	[p. 12, Section 5.3 Time-Shifting Computing, Block P-031]	Opportunistic-quota functions can be deferred to off-peak hours. The CPU consumption of reserved and opportunistic functions almost exactly complements each other, showing that XFaaS schedules opportunistic functions during off...	说明可延迟工作负载为何适合 serverless 平台优化。

Detailed answer

不适合的情况主要是交互关键路径和严格低延迟场景。论文明确说 XFaaS 很少处理要求亚秒级响应的用户交互请求, 例如 newsfeed display、search result ranking、video playback; 这些请求在 Meta 通常由长期运行服务处理, 因为 serverless functions 在这些场景中没有明显优势。[p. 5, Section 3.2 Workload Examples, Block P-016]

论文还解释了原因：serverless 常被称赞的两个优势是按需付费/无需预先容量规划，以及平台自动化部署；但对于 Meta 级别的用户体验，仍需要精细容量规划，而自动化部署本身也可以由 Conveyor 这样的系统完成。[p. 5, Section 3.2 Workload Examples, Block P-017]

适合的情况是异步、事件驱动和可延迟工作。论文列出的典型工作负载包括接受好友请求后异步生成推荐、日志事件处理、规则触发的 [p. 5, Section 3.2 Workload Examples, Block P-018]

更一般地，理想候选通常由队列、定时器、存储、编排或事件桥触发，而不是由直接 HTTP/RPC 触发；它们更可能没有用户在同步等待结果，因此平台可以用截止时间、关键性和机会式配额把执行平滑到低峰期。[p. 3, Section 2.2 Spiky Load, Block P-012] [p. 12, Section 5.3 Time-Shifting Computing, Block P-031]

Blocks to reread

- [p. 5, Section 3.2 Workload Examples, Block P-016]■- [p. 5, Section 3.2 Workload Examples, Block P-017]■- [p. 5, Section 3.2 Workload Examples, Block P-018]■- [p. 12, Section 5.3 Time-Shifting Computing, Block P-031]

Uncertainty

The paper gives strong evidence from Meta's private-cloud context. Applying the same fit criteria to a public-cloud application requires checking whether that application has similar latency, trigger, trust, isolation, and capacity characteristics.

Q3. What characteristics would make an application ideal for serverless deployment?

Relevance judgment

Partially relevant.

Short answer

本文没有给出一个正式的“ideal serverless application”定义，但可以从 XFaaS 的设计和 workload 归纳出理想特征：事件驱动、异步、不在亚秒级用户交互关键路径上；可表达完成时限、关键性和资源配额；能容忍 [p. 3, Section 2.2 Spiky Load, Block P-012] [p. 4, Section 2.4 Terminology, Block P-014] [p. 5, Section 3.2 Workload Examples, Block P-018] [p. 14, Section 8 Conclusion, Block P-041]

Evidence table

Claim	Citation	Evidence from paper	How it supports the answer
理想 serverless 工作负载通常不是直接 RPC/HTTP 同步请求，而是更能容忍延迟的事件触发工作。	[p. 3, Section 2.2 Spiky Load, Block P-012]	XFaaS clients submit function calls in a highly spiky manner. One key insight is that some functions do not need immediate execution; functions triggered by queues, timers, storage, orchestration, and event-bridge workflows tend to be asynchronous.	论文把触发方式和延迟容忍度联系起来。
理想函数应能声明运行时、关键性、开始时间、完成时限、配额、并发限制和重试策略。	[p. 4, Section 2.4 Terminology, Block P-014]	A function has configurable attributes such as name, arguments, runtime, criticality, execution start time, completion deadline, resource quota, concurrency limit, and retry policy. Completion deadlines can range from seconds to minutes.	这些属性让平台进行调度、限流和成本优化。
异步推荐、日志处理、自动化、通知和数据转换体现了适合 XFaaS 的应用形态。	[p. 5, Section 3.2 Workload Examples, Block P-018]	Good XFaaS examples include asynchronous recommendations after user events, log processing that is not on the critical path but still has an SLO, productivity automation triggered by events, notification campaigns, and Morphing...	这些例子共同体现了事件驱动和非同步关键路径特征。
理想应用还要适配 worker 内存和局部性约束。	[p. 9, Section 4.5.2 Locality Groups, Block P-026]	A true universal worker is infeasible because keeping every function's JIT code in every worker's memory would exceed memory capacity, and memory-hungry functions may also exhaust memory. XFaaS partitions functions and workers...	函数规模和内存行为会影响能否高效部署。
XFaaS 的结论强调无冷启动、低峰执行和下游服务影响。	[p. 14, Section 8 Conclusion, Block P-041]	XFaaS enables workers to serve requests for almost any function without cold start through proactive code deployment, cooperative JIT, cross-function runtime sharing, and locality groups. It also postpones delay-tolerant computation...	总结了理想应用应配合的平台优化方向。

Detailed answer

理想 serverless 应用首先应是事件驱动且异步的。论文认为，队列、定时器、存储、编排和事件桥触发的函数通常比直接 HTTP/RPC 触发函数更能容忍延迟，因为它们不一定对应一个正在等待即时响应的交互用户。[p. 3, Section 2.2 Spiky Load, Block P-012] [p. 13, Section 6 XFaaS and Public Cloud, Block P-037]

其次，它应能把业务需求转化为平台可调度的属性：完成时限、关键性、资源配额、并发限制、重试策略和可选的未来开始时间。这样 [p. 4, Section 2.4 Terminology, Block P-014] [p. 14, Section 6 XFaaS and Public Cloud, Block P-038]

第三，它应允许平台做成本优化。若应用的工作能通过机会式配额被移到低峰期，XFaaS 就能让机会式函数和预留函数的 CPU 消耗互补，从而降低峰值容量需求。 [p. 12, Section 5.3 Time-Shifting Computing, Block P-031]

第四，它不应无控制地放大下游压力。理想的 serverless 应用要么对下游服务影响小，要么能配合背压和并发控制；XFaaS 的生产事故表明，自动减速和排队可在下游恢复后继续处理积压。 [p. 12, Section 5.5 Protecting Downstream Services, Block P-033] [p. 13, Section 5.5 Protecting Downstream Services, Block P-034]

Blocks to reread

- [p. 3, Section 2.2 Spiky Load, Block P-012]■- [p. 4, Section 2.4 Terminology, Block P-014]■- [p. 5, Section 3.2 Workload Examples, Block P-018]■- [p. 14, Section 8 Conclusion, Block P-041]

Uncertainty

This answer is an interpretation grounded in the paper's workload examples and system design, because the paper does not explicitly define a universal checklist for an ideal serverless application.

Glossary

Function-as-a-Service / FaaS: 函数即服务 / FaaS - 本文核心领域术语。

serverless computing: 无服务器计算 - 按无服务器语境翻译。

function call / invocation: 函数调用 - 在本文中二者基本同义。

cold start: 冷启动 - 函数初始化路径带来的延迟和资源开销。

universal worker: 通用工作节点 - 近似每个 worker 可立即执行任意函数的效果。

worker: 工作节点 - 运行函数的物理服务器。

runtime: 运行时 - PHP、Python 等函数执行环境。

namespace: 命名空间 - 隔离的 worker 池和函数集合。

locality group: 局部性组 - 限制 worker 执行的函数子集以降低内存压力。

cooperative JIT compilation: 协作式 JIT 编译 - 跨 worker 共享 profiling/JIT 信息。

time-shift computing: 时间转移计算 - 把可延迟函数推迟到低峰执行。

reserved quota: 预留配额 - 保证容量的配额类型。

opportunistic quota: 机会式配额 - 可推迟到低峰执行的低成本配额。

back-pressure: 背压 - 下游服务过载时返回给 XFaaS 的限速信号。

adaptive concurrency control: 自适应并发控制 - 根据下游健康状态调节函数执行速率。

downstream service: 下游服务 - 函数调用依赖的数据库、缓存等服务。

SLO: 服务级目标 / SLO - 以完成时限或可用性表达的目标。

criticality: 关键性 - 容量不足时决定执行优先级的属性。

AUTH-001 | Title | Original PDF p. 1

Alireza Sahraei, Soteris Demetriou, Amirali Sobhgol, Haoran Zhang, Abhigna Nagaraja, Neeraj Pathak, Girish Joshi, Carla Souza, Bo Huang, Wyatt Cook, Andrii Golovei, Pradeep Venkat, Andrew McFague, Dimitrios Skarlatos, Vipul Patel, Ravinder Thind, Ernesto Gonzalez, Yun Jin, and Chunqiang Tang. Meta Platforms, Imperial College London, University of Pennsylvania, Carnegie Mellon University.

作者包括 Alireza Sahraei、Soteris Demetriou、Amirali Sobhgol、Haoran Zhang、Abhigna Nagaraja、Neeraj Pathak、Girish Joshi、Carla Souza、Bo Huang、Wyatt Cook、Andrii Golovei、Pradeep Venkat、Andrew McFague、Dimitrios Skarlatos、Vipul Patel、Ravinder Thind、Ernesto Gonzalez、Yun Jin 和 Chunqiang Tang；机构包括 Meta Platforms、Imperial College London、University of Pennsylvania 和 Carnegie Mellon University。

ABS-001 | Abstract | Original PDF p. 1

Function-as-a-Service (FaaS) has become a popular programming paradigm in Serverless Computing. As the responsibility of resource provisioning shifts from users to cloud providers, the ease of use of FaaS for users may come at the expense of extra hardware costs for cloud providers. This paper presents XFaaS in Meta's hyperscale private cloud, which processes trillions of function calls per day on more than 100,000 servers. XFaaS uses optimizations that help achieve a daily average CPU utilization of 66%, including approximating universal workers, deferring delay-tolerant functions to off-peak hours, globally dispatching calls across regions, and using TCP-like congestion control for downstream services.

函数即服务（FaaS）已经成为无服务器计算中的常见编程范式。随着资源供给责任从用户转移到云服务提供方，FaaS 对用户的易用性可能转化为云服务提供方额外的硬件成本。本文介绍 Meta 超大规模私有云中的 XFaaS：它每天处理数万亿次函数调用，运行在超过 100,000 台服务器之上。XFaaS 通过一组优化达到 66% 的日均 CPU 利用率，包括近似“通用工作节点”、把可延迟函数推迟到低峰期、跨区域全局分发调用，以及用类似 TCP 的拥塞控制机制保护下游服务。

P-001 | 1 Introduction | Original PDF p. 1

With FaaS, developers create individual functions and upload them to a provider platform, where the platform provisions resources and executes functions in response to external events. This frees developers from managing VMs or containers.

在 FaaS

中，开发者创建单个函数并上传到云服务提供方的平台；平台负责资源供给，并在外部事件触发时执行函数。这让开发者不

P-002 | 1 Introduction | Original PDF p. 1

The ease of use of FaaS may come at the expense of extra hardware costs for cloud providers. Before FaaS, users carried the cost of underutilized VMs; with FaaS, cloud providers bear the cost of over-provisioned resources.

FaaS 的易用性可能以云服务提供方承担额外硬件成本为代价。在 FaaS 之前，用户承担为间歇性调用而过度配置、但利用率不足的 VM 成本；在 FaaS 模式下，过度供给资源的成本转移给云服务提供方。

P-003 | 1 Introduction | Original PDF p. 1

Meta operates XFaaS in a hyperscale private cloud. XFaaS processes trillions of function calls per day on more than 100,000 servers across tens of datacenter regions, making hardware-cost reduction a top priority.

Meta 在超大规模私有云中运行 XFaaS。XFaaS

每天处理数万亿次函数调用，分布在数十个数据中心区域、超过 100,000 台服务器上，因此降低硬件成本成为首要目标。

FIGCAP-001 | 1.1 Risk of Extra Hardware Costs | Original PDF p. 2

Figure 1. Lifecycle of a function. The lifecycle includes VM startup, fetching code, container initialization, runtime startup, library and code loading, optional JIT compilation, invocation, and later container/VM shutdown.

图 1. 函数生命周期。生命周期包括启动

VM、获取代码、初始化容器、启动运行时、加载库和函数代码、可选 JIT 编译、调用函数，以及之后关闭容器/VM。

P-004 | 1.1 Risk of Extra Hardware Costs | Original PDF p. 2

Lengthy cold-start time is a significant FaaS challenge because initialization and shutdown steps are overheads borne by the provider, while only invocation does chargeable work. If the idle wait is too long, containers waste resources; if too short, the next request pays initialization overhead again.

较长的冷启动时间是 FaaS

的重要挑战，因为初始化和关闭步骤都属于由平台承担的开销，只有实际调用阶段执行可计费工作。如果空闲等待时间过

P-005 | 1.1 Risk of Extra Hardware Costs | Original PDF p. 2

High load variance creates a tradeoff between extra hardware cost from over-provisioning and overload during surges. XFaaS sees a peak-to-trough ratio as high as 4.3, so provisioning for peak demand would create severe waste.

负载高度波动会带来取舍：若按峰值配置，会产生额外硬件成本；若配置不足，负载激增时系统会过载。XFaaS 的峰谷比最高达到 4.3，因此按峰值配置会造成严重浪费。

P-006 | 1.1 Risk of Extra Hardware Costs | Original PDF p. 2

FaaS functions can overload downstream services. Meta experienced outages when spikes from non-user-facing functions overloaded a social-graph database. Static per-function concurrency limits are insufficient because limits that are too low waste resources, while limits that are too high can increase error rates for online services.

FaaS 函数还可能压垮下游服务。Meta

曾遇到非用户直接访问函数的调用峰值压垮社交图数据库，从而导致面向用户的在线服务错误率升高。静态的单函数并发限制会消耗 FaaS 平台和数据库资源，限制过高又可能让在线服务错误率升高。

P-007 | 1.2 Solutions for Extra Hardware Costs | Original PDF p. 2

XFaaS tries to approximate a universal worker: every worker can instantly execute every function without cold-start overhead. It pushes code to local SSDs, keeps language runtimes running, runs multiple functions in one Linux process in a trusted private cloud, uses cooperative JIT compilation, and restricts a server to a subset of functions to improve JIT cache hits.

XFaaS

试图近似“通用工作节点”：每个工作节点都能无冷启动开销地立即执行任意函数。它把代码主动推送到本地 SSD，保持语言运行时持续运行；在可信私有云中让多个函数并发运行在同一个 Linux 进程内；使用协作式 JIT 编译；并把某台服务器接收的函数限制在一个子集内，以提高 JIT 代码缓存命中率。

P-008 | 1.2 Solutions for Extra Hardware Costs | Original PDF p. 2

To reduce peak hardware cost, XFaaS intentionally provisions less capacity than peak demand and manages overload through time-shift computing, deadlines and criticality, cross-region dispatch, per-function quotas, throttling, and caller-specified future start times.

为了降低峰值硬件成本, XFaaS

有意不按峰值需求配置容量, 而是通过时间转移计算、完成时限和关键性、跨区域分发、单函数配额、限流以及调用方指

P-009 | 1.2 Solutions for Extra Hardware Costs | Original PDF p. 3

XFaaS addresses downstream overload through adaptive concurrency control. It uses a global resource isolation and management system to collect metrics across systems and allows downstream services to send back-pressure signals; XFaaS responds with a TCP-like congestion-control mechanism.

XFaaS

通过自适应并发控制处理下游过载。它使用全局资源隔离与管理系统跨系统收集指标，并允许下游服务发送背压信号；XFaaS 随后用类似 TCP 的拥塞控制机制调节函数执行节奏。

P-010 | 1.2 Contributions | Original PDF p. 3

The paper's contributions are production evidence about FaaS hardware cost, techniques approximating universal workers, techniques for load-spike management such as time-shift computing and cross-region dispatch, and adaptive concurrency control to protect downstream services.

本文贡献包括：用生产数据说明 FaaS

易用性是否会带来平台硬件成本；提出近似通用工作节点的一组技术；提出通过时间转移计算、跨区域分发等方式管理负载

P-011 | 2.1 Rapid Growth of FaaS in Our Private Cloud | Original PDF p. 3

Daily function invocations in XFaaS increased 50 times over five years and now total trillions per day. The hyperscale of XFaaS requires efficient hardware use, especially under spiky loads.

XFaaS 的每日函数调用量在五年内增长了 50

倍，目前每天达到数万亿次。这样的超大规模要求系统高效使用硬件，尤其是在应对尖峰负载时。

P-012 | 2.2 Spiky Load | Original PDF p. 3

XFaaS clients submit function calls in a highly spiky manner. One key insight is that some functions do not need immediate execution; functions triggered by queues, timers, storage, orchestration, and event-bridge workflows tend to be more delay-tolerant than direct RPC/HTTP functions.

XFaaS

客户端提交函数调用的方式高度尖峰化。关键洞察之一是，并非所有函数都需要立即执行；由队列、定时器、存储、编排和 RPC/HTTP 触发的函数更能容忍延迟。

P-013 | 2.3 Datacenters and Uneven Hardware Distribution | Original PDF p. 4

XFaaS must balance regional locality and cross-region load balancing. Within a region, hardware is largely fungible, but cross-region bandwidth is lower and latency is much higher, so cross-region dispatch adds complexity.

XFaaS

必须在区域局部性和跨区域负载均衡之间取舍。同一区域内硬件基本可以互换，但跨区域带宽更低、延迟高得多，因此跨区

P-014 | 2.4 Terminology | Original PDF p. 4

A function has configurable attributes such as name, arguments, runtime, criticality, execution start time, completion deadline, resource quota, concurrency limit, and retry policy. Completion deadlines can range from seconds to 24 hours, and delay-tolerant functions can be postponed to off-peak hours.

函数具有多个可配置属性，包括名称、参数、运行时、关键性、执行开始时间、完成时限、资源配额、并发限制和重试策略。重试策略可以是 24 小时不等；当容量不足时，可容忍延迟的函数可以被推迟到低峰期执行。

P-015 | 3 Diverse Workloads | Original PDF p. 4

XFaaS supports workloads from thousands of teams, with runtimes including PHP, Python, Erlang, Haskell, and C++, and triggers including queues, orchestration workflows, timers, storage, data streams, and event bridges.

XFaaS 支持来自数千个团队的多样化工作负载，运行时包括 PHP、Python、Erlang、Haskell 和 C++，触发方式包括队列、编排工作流、定时器、存储、数据流和事件桥。

TABLECAP-001 | 3.1 Function Categories | Original PDF p. 5

Table 1. Breakdown of functions by categories: queue-triggered functions are 89% of functions and 86% of compute usage; event-triggered functions are 8% of functions and 85% of function calls; timer-triggered functions are 3% of functions.

表 1. 按类别划分的函数：队列触发函数占函数数量的 89%、计算用量的 86%；事件触发函数占函数数量的 8%、函数调用量的 85%；定时触发函数占函数数量的 3%。

P-016 | 3.2 Workload Examples | Original PDF p. 5

XFaaS workloads seldom handle user-facing interactive requests demanding sub-second response times, such as newsfeed display, search-result ranking, or video playback. At Meta, those requests are traditionally handled by long-running services because serverless functions offer no significant advantages in these scenarios.

XFaaS 工作负载很少处理要求亚秒级响应的用户交互请求，例如信息流展示、搜索结果排序或视频播放。在 Meta，这类请求通常由长期运行的服务处理，因为无服务器函数在这些场景中并没有显著优势。

P-017 | 3.2 Workload Examples | Original PDF p. 5

The paper identifies two common benefits of serverless functions: pay-as-you-go with no upfront capacity planning, and streamlined deployment where developers write code while the platform handles deployment. For user-facing requests needing sub-second latency, the first benefit loses relevance because meticulous capacity planning is needed for Meta-scale user experience; the second benefit can be achieved with Meta's deployment automation.

论文指出无服务器函数通常有两个主要优势：按需付费且无需预先容量规划；开发者只写代码，由平台自动处理部署。但在 Meta 场景下不再关键，因为为了数十亿用户的体验仍需精细容量规划；第二个优势也可以通过 Meta 的自动化部署系统实现。

P-018 | 3.2 Workload Examples | Original PDF p. 5

Good XFaaS examples include asynchronous recommendations after user events, log processing that is not on the critical path but still has an SLO, productivity automation triggered by events, notification campaigns, and Morphing Framework functions that perform custom data transformation across stores.

适合 XFaaS 的例子包括：用户事件之后异步生成推荐；不在交互关键路径上、但仍有 SLO 的日志处理；由事件触发的生产力自动化；通知活动调度；以及在不同数据存储之间执行自定义数据转换的 Morphing Framework 函数。

P-019 | 3.3 Workload Resource Usage | Original PDF p. 5

Resource consumption varies widely. CPU per call ranges from 0.37 MIPS at P10 to 1,064,280 MIPS at P99; 33% of calls finish within one second, 94% within 60 seconds, and 1% exceed five minutes.

资源消耗差异很大。单次调用 CPU 用量从 P10 的 0.37 MIPS 到 P99 的 1,064,280 MIPS 不等；33% 的调用在 1 秒内完成，94% 在 60 秒内完成，1% 超过 5 分钟。

P-020 | 4.1 Overview | Original PDF p. 7

XFaaS's architecture includes submitters, durable queues, schedulers, worker load balancers, worker pools, configuration management, rate limiting, locality optimization, utilization control, and a global traffic conductor.

XFaaS

架构包括提交器、持久队列、调度器、工作节点负载均衡器、工作节点池、配置管理、限速、局部性优化、利用率控制和

P-021 | 4.2 Submitter | Original PDF p. 7

Before sending a call, the submitter obtains the function's metadata and determines the target region and durable queue. It may route to the caller's preferred region or to another region for load balancing.

提交器在发送调用之前获取函数元数据，并决定目标区域和持久队列。它可以把调用路由到调用方偏好的区域，也可以为了负载均衡路由到其他区域。

P-022 | 4.3 DurableQ | Original PDF p. 7

DurableQ provides at-least-once semantics. Calls are stored persistently and become available for retry if the scheduler fails to acknowledge them after dispatch.

DurableQ

提供至少一次语义。调用会被持久保存；如果调度器分发后没有确认，该调用会重新可见，以便其他调度器获取并重试。

P-023 | 4.4 Scheduler | Original PDF p. 8

A scheduler orders function calls by criticality, deadline, and capacity quota. It reads per-function FuncBuffers and produces a RunQ that is dispatched for execution.

调度器根据关键性、截止时间和容量配额对函数调用排序。它读取每个函数的 FuncBuffer，并生成用于分发执行的 RunQ。

P-024 | 4.5 WorkerLB and Worker Pool | Original PDF p. 8

Worker load balancers dispatch calls from RunQ to worker pools. Workers keep runtimes alive and execute multiple functions concurrently in one process, enabled by strong trust and isolation assumptions in the private cloud.

工作节点负载均衡器把 RunQ

中的调用分发到工作节点池。工作节点保持运行时常驻，并在一个进程中并发执行多个函数；这依赖私有云中的强信任和

P-025 | 4.5.1 Cooperative JIT Compilation | Original PDF p. 9

Cooperative JIT compilation lets one server collect profiling data for a new function version and propagate it to other servers, so those servers can pre-compile the function before receiving calls.

协作式 JIT 编译让一台服务器为新函数版本收集 profiling

数据，并把这些数据传播给其他服务器，使它们在收到调用之前就能预编译函数。

P-026 | 4.5.2 Locality Groups | Original PDF p. 9

A true universal worker is infeasible because keeping every function's JIT code in every worker's memory would exceed memory capacity, and memory-hungry functions may also exhaust memory. XFaaS partitions functions and workers into locality groups so only calls for a subset of functions are dispatched to a worker.

真正的通用工作节点并不可行，因为把每个函数的 JIT

代码都保存在每个工作节点内存中会超出内存容量；而内存密集型函数也可能耗尽内存。XFaaS 将函数和工作节点划分为局部性组，使某个工作节点只接收一部分函数的调用。

P-027 | 4.6 Quotas and Utilization Control | Original PDF p. 10

XFaaS supports reserved quota and opportunistic quota. Reserved quota provides guaranteed capacity; opportunistic quota can be delayed until off-peak hours. This creates a direct tradeoff between latency guarantees and lower hardware cost.

XFaaS

支持预留配额和机会式配额。预留配额提供保证容量；机会式配额可以被推迟到低峰时段执行。这形成了延迟保证与较低

P-028 | 4.7 Adaptive Concurrency Control | Original PDF p. 10

For a function's RPS limit r and execution time p , the number of running instances can reach $R = r \times p$. If p is large, R can be large and cause too many concurrent calls to a downstream service, so XFaaS also supports concurrency limiting and back-pressure-aware rate control.

对于 RPS 限制为 r 、执行时间为 p 的函数，运行实例数可能达到 $R = r \times p$ 。如果 p 很大， R 也会很大，从而对下游服务产生过多并发调用。因此 XFaaS 还支持并发限制和感知背压的速率控制。

P-029 | 5.1 Overall CPU Utilization | Original PDF p. 11

XFaaS achieves a daily average CPU utilization of 66%, and regional CPU utilization stays in a narrow band despite uneven hardware distribution. The peak-to-trough ratio of CPU utilization is only 1.4x, much lower than the 4.3x ratio for received calls.

XFaaS 达到 66% 的日均 CPU 利用率；尽管硬件分布不均，不同区域的 CPU 利用率仍保持在较窄范围内。CPU 利用率的峰谷比只有 1.4x，显著低于接收调用的 4.3x 峰谷比。

P-030 | 5.2 Locality Group and Memory Consumption | Original PDF p. 11

A production experiment found that locality groups reduced worker memory consumption by 11.8% at P50 and 11.4% at P95. The results show that locality groups manage memory pressure while approximating universal workers.

生产实验发现，局部性组使工作节点内存消耗在 P50 降低 11.8%、在 P95 降低 11.4%。结果表明，局部性组能够管理内存压力，同时近似通用工作节点效果。

P-031 | 5.3 Time-Shifting Computing | Original PDF p. 12

Opportunistic-quota functions can be deferred to off-peak hours. The CPU consumption of reserved and opportunistic functions almost exactly complements each other, showing that XFaaS schedules opportunistic functions during off-peak hours and can reduce peak capacity needs.

使用机会式配额的函数可以被推迟到低峰时段。预留函数和机会式函数的 CPU 消耗几乎互补，说明 XFaaS 能把机会式函数调度到低峰执行，并降低峰值容量需求。

P-032 | 5.4 Cooperative JIT Compilation | Original PDF p. 12

With cooperative JIT compilation, a worker reached maximum RPS in three minutes after restart; without supplied JIT profiling data, it took 21 minutes. This highlights the importance of cooperative JIT under frequent function-code updates.

使用协作式 JIT 编译时，工作节点重启后 3 分钟达到最大 RPS；如果没有提供 JIT profiling 数据，则需要 21 分钟。这说明在频繁推送函数代码更新的情况下，协作式 JIT 非常重要。

P-033 | 5.5 Protecting Downstream Services | Original PDF p. 12

During a WTCache incident, XFaaS slowed calls to functions that would further load WTCache, stored pending calls in DurableQs, and gradually increased execution rates after recovery; the backlog was processed within two hours.

在一次 WTCache 事故中，XFaaS 放慢了会进一步增加 WTCache 负载的函数调用，把待处理调用保存在 DurableQ 中，并在 WTCache 恢复后逐步提高执行速率；积压任务在两小时内处理完毕。

P-034 | 5.5 Protecting Downstream Services | Original PDF p. 13

During a TAO migration incident, XFaaS automatically slowed as many as 200 unique functions and reduced traffic to TAO in the impacted region by about 40%, limiting user impact without manual intervention.

在一次 TAO 迁移事故中，XFaaS 自动放慢多达 200 个不同函数的执行，使受影响区域流向 TAO 的总流量减少约 40%，并在无需人工干预的情况下限制了用户影响。

P-035 | 6 XFaaS and Public Cloud | Original PDF p. 13

XFaaS prioritizes local-region execution but can dispatch globally across regions for load balancing. This adds global coordination complexity, but the authors argue it may be relevant to public clouds as providers expand to many regions.

XFaaS

优先在本地区域执行，但必要时可以跨区域全局分发以实现负载均衡。这增加了全局协调复杂性，但作者认为，随着公有云

P-036 | 6 XFaaS and Public Cloud | Original PDF p. 13

The authors argue that FaaS platforms should optimize resource utilization and throughput, not only call latency. If a platform focuses mainly on latency, it may keep VMs idle for long periods after invocation and over-provision for peak demand, leading to low hardware utilization.

作者认为, FaaS

平台不应只关注调用延迟, 还应重视资源利用率和吞吐量。如果平台主要优化延迟, 就可能在调用后长时间保持 VM 空闲, 并按峰值需求过度配置硬件, 从而导致硬件利用率低。

P-037 | 6 XFaaS and Public Cloud | Original PDF p. 13

Industry workloads also contain many calls triggered by queues, events, storage, timers, and orchestration workflows. Some may have response expectations of a few seconds, but many non-RPC events are less likely to involve an interactive user waiting for an immediate response.

行业中的 FaaS

工作负载也包含大量由队列、事件、存储、定时器和编排工作流触发的调用。其中一部分可能有数秒级响应预期，但许多 RPC 事件不像 HTTP/RPC 调用那样直接对应一个正在等待即时响应的交互用户。

P-038 | 6 XFaaS and Public Cloud | Original PDF p. 14

Techniques that may apply to public clouds include future execution start times, user-chosen SLOs expressed as completion deadlines, and criticality levels so critical functions run first when capacity is low.

可能适用于公有云的技术包括：允许调用方指定未来执行开始时间；允许函数所有者用完成截止时间表达 SLO；以及设置关键性等级，使容量不足时关键函数优先执行。

P-039 | 6 XFaaS and Public Cloud | Original PDF p. 14

Although public clouds cannot run functions from different users in the same Linux process as XFaaS does, large public-cloud customers may adopt XFaaS-like approaches inside virtual private clouds because a small fraction of teams can consume most capacity.

尽管公有云不能像 XFaaS 那样把不同用户的函数运行在同一个 Linux 进程内，但大型公有云客户可能在其虚拟私有云中采用类似 XFaaS 的方法，因为少数团队往往消耗了大部分容量。

P-040 | 7 Related Work | Original PDF p. 14

Prior work studied serverless surveys, function scheduling, cold-start reduction, industry systems, and workload characterization. XFaaS differs by combining deferred delay-tolerant execution, precompiled runtimes, locality groups, and back-pressure mechanisms that consider downstream services.

已有工作研究了无服务器综述、函数调度、冷启动降低、工业系统和工作负载特征。XFaaS 的不同之处在于组合了可延迟执行、预编译运行时、局部性组，以及考虑下游服务的背压机制。

P-041 | 8 Conclusion | Original PDF p. 14

XFaaS enables workers to serve requests for almost any function without cold start through proactive code deployment, cooperative JIT, cross-function runtime sharing, and locality groups. It also postpones delay-tolerant computation to off-peak hours and argues that FaaS workload management must consider downstream-service impact.

XFaaS 通过主动代码部署、协作式

JIT、跨函数运行时共享和局部性组，使工作节点几乎能够无冷启动地服务任意函数请求。它还把可容忍延迟的计算推迟到 FaaS 工作负载管理必须考虑对下游服务的影响。

ACK-001 | Acknowledgments | Original PDF p. 14

The paper presents a decade of work by several teams at Meta, including Async, FOQS, Web Foundation, Serverless Workflow, HHVM, and CEA.

本文呈现了 Meta 多个团队十年工作的成果，包括 Async、FOQS、Web Foundation、Serverless Workflow、HHVM 和 CEA。

REF-001 | References | Original PDF p. 15

References [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Fire-cracker: Lightweight virtualization for serverless applications. In NSDI, volume 20, pages 419–434, 2020. [2] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. SAND: Towards high-performance serverless computing. In 2018 USENIX Annual Technical Conference, pages 923–935, 2018. [3] Apache OpenWhisk. Accessed on April 4, 2023. [4] AWS Lambda. Accessed on April 4, 2023. [5] Azure Functions. Accessed on April 4, 2023. [6] Je■ Barr. New - accelerate your lambda functions with lambda snap-start, 2022. Accessed on April 4, 2023. [7] David Elzio/t Bell. Looking back at the bell-la padula model. In 21st Annual Computer Security Applications Conference (ACSAC'05), pages 15–pp. IEEE, 2005. [8] Sol Boucher, Anuj Kalia, David G Andersen, and Michael Kaminsky. Pu/t_ting the "micro" back in microservice. In 2018 USENIX Annual Technical Conference, pages 645–650, 2018. [9] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkat Venkataramani. TAO: Facebook's Distributed Data Store for the Social Graph. In Proceedings of the 2013 USENIX Annual Technical Conference, pages 49–60, 2013. [10] Marc Brooker, Mike Danilov, Chris Greenwood, and Phil Piwonka. On-demand Container Loading in AWS Lambda. In 2023 USENIX Annual Technical Conference, pages 315–328, 2023. [11] James Cadden, /T_homas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: skip redundant paths to make serverless fast. In Proceedings of the Fi/f_teenth European Conference on Computer Systems, pages 1–15, 2020. [12] Guoqiang Jerry Chen, Janet L Wiener, Shridhar Iyer, Anshul Jaiswal, Ran Lei, Nikhil Simha, Wei Wang, Kevin Wilfong, Tim Williamson, and Serhat Yilmaz. Realtime Data Processing at Facebook. In Proceedings of the 2016 International Conference on Management of Data, pages 1087–1098, 2016. [13] Simon Eismann, Joel Scheuner, Erwin Van Eyk, Maximilian Schwinger, Johannes Grohmann, Nikolas Herbst, Cristina L Abad, and Alexandru Iosup. Serverless applications: Why, when, and how? IEEE So/f_tware, 38(1):32–39, 2020. [14] Marius Eriksen, Kaushik Veeraraghavan, Yusuf Abdulghani, Andrew Birchall, Po-Yen Chou, Richard Cornew, Adela Kabiljo, Ranjith Kumar S, Maroo Lieuw, Justin Meza, Sco/t_t Michelson, /T_homas Rohloff, Hayley Russell, Je■ Qin, and Chunqiang Tang. Global Capacity Management with Flux. In Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation, 2023. [15] Jason Flinn, Xianzheng Dou, Arushi Aggarwal, Alex Boyko, Francois Richard, Eric Sun, Wendy Tobagus, Nick Wolchko, and Fang Zhou. Owl: Scale and flexibility in distribution of hot content. In 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), pages 1–15, 2022. [16] Alim Ul Gias and Giuliano Casale. Cocoa: Cold start aware capacity planning for function-as-a-service platforms. In 2020 28th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), pages 1–8. IEEE, 2020. [17] Google Cloud Functions. Accessed on April 4, 2023. [18] Boris Grubic, Yang Wang, Tyler Petrochko, Ran Yaniv, Brad Jones, David Callies, Ma/t_t Clarke-Lauer, Dan Kelley, Soteris Demetriou, Kenny Yu, and Chunqiang Tang. Conveyor: One-Tool-Fits-All Contin- uous So/f_tware Deployment at Meta. In Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation, 2023. [19] Katia Gil Guzman. How to Use Azure Queue-Triggered Functions and Why, December 2020. <https://medium.com/swlh/how-to-use-azure-queue-triggered-functions-and-why-7f651c9d3f8c>. [20] Hassan B Hassan, Saman A Barakat, and /Q_usay I Sarhan. Survey on serverless computing. Journal of Cloud Computing, 10(1):1–29, 2021. [21] Joseph M Hellerstein, Jose Faleiro, Joseph E Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. Serverless computing: One step forward, two steps back. arXiv preprint arXiv:1812.03651, 2018. [22] Sco/t_t Hendrickson, Stephen Sturdevant, Tyler Harter, Venkateshwaran Venkataramani, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Serverless Computation with OpenLambda. In 8th USENIX workshop on hot topics in cloud computing (HotCloud 16), 2016. [23] HHVM. <https://hhvm.com>. [24] Hive. <https://hive.apache.org>. [25] Zhipeng Jia and Emme/t_t Witchel. Boki: Stateful serverless computing with shared logs. In Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles, pages 691–707, 2021. [26] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. Cloud programming simplifi_ed: A berkeley view on serverless computing. arXiv preprint arXiv:1902.03383, 2019. [27] Kostis Ka■es, Neeraja J Yadwadkar, and Christos Kozyrakis. Central- ized core-granular scheduling for serverless functions. In Proceedings of the ACM symposium on cloud computing, pages 158–164, 2019. [28] Kostis Ka■es, Neeraja J Yadwadkar, and Christos Kozyrakis. Practi- cal scheduling for real-world serverless computing. arXiv preprint arXiv:2111.07226, 2021. [29] Zijun Li, Linsong Guo, Jiagan Cheng, /Q_uan Chen, BingSheng He, and Minyi Guo. The serverless computing survey: A technical primer for

design architecture. *ACM Computing Surveys (CSUR)*, 54(10s):1–34, 2022. [30] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 303–320, 2022. [31] Anupama Mampage, Shanika Karunasekera, and Rajkumar Buyya. A holistic view on resource management in serverless computing environments: Taxonomy and future directions. *ACM Computing Surveys (CSUR)*, 54(11s):1–36, 2022. [32] Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104, 2001. [33] Anup Mohan, Harshad S Sane, Kshitij Doshi, Saikrishna Edupuganti, Naren Nayak, and Vadim Sukhominov. Agile cold starts for scalable serverless. *HotCloud*, 2019(10.5555):3357034–3357060, 2019. [34] Edward Oakes, Leon Yang, Kevin Houck, Tyler Harter, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Pipsqueak: Lean lambda with large libraries. In *2017 IEEE 37th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, pages 395–400. IEEE, 2017. [35] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. SOCK: Rapid task provisioning with serverless-optimized containers. In *2018 USENIX Annual Technical Conference*, pages 57–70, 2018. [36] Guilherme O/t_toni. HHVM JIT: A Prof/ile-guided, Region-based Compiler for PHP and Hack. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 151–165, 2018. [37] Guilherme O/t_toni and Bin Liu. HHVM jump-start: Boosting both warmup and steady-state performance at scale. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 245 pages 340–350. IEEE, 2021. [38] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 753–767, 2022. [39] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *USENIX Annual Technical Conference*, 2020. [40] Chunqiang Tang, /T_hawan Kooburat, Pradeep Venkatachalam, Akshay Chander, Zhe Wen, Aravind Narayanan, Patrick Dowell, and Robert Karl. Holistic Configuration Management at Facebook. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 328–343, 2015. [41] Ali Tariq, Austin Pahl, Sharat Nimmagadda, Eric Rozner, and Siddharth Lanka. Sequoia: Enabling quality-of-service in serverless computing. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 311–327, 2020. [42] Ashish /T_husoo, Zheng Shao, Suresh Anthony, Dhruba Borthakur, Namit Jain, Joydeep Sen Sarma, Raghotham Murthy, and Hao Liu. Data warehousing and analytics infrastructure at facebook. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 1013–1020, 2010. [43] Ao Wang, Shuai Chang, Huangshi Tian, Hongqi Wang, Haoran Yang, Huiba Li, Rui Du, and Yue Cheng. FaaSNet: Scalable and Fast Provisioning of Custom Serverless Container Runtimes at Alibaba Cloud Function Compute. In *USENIX Annual Technical Conference*, 2021. [44] Kai-Ting Amy Wang, Rayson Ho, and Peng Wu. Replayable execution optimized for page sharing for a managed runtime environment. In *Proceedings of the Fourteenth EuroSys Conference 2019*, pages 1–16, 2019. [45] Liang Wang, Mengyuan Li, Yinqian Zhang, /T_homas Ristenpart, and Michael Swi/f_t. Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference*, pages 133–146, 2018. [46] Ethan G Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. The true cost of containing: A gvisor case study. In *HotCloud*, 2019. [47] Tianyi Yu, Qingyuan Liu, Dong Du, Yubin Xia, Binyu Zang, Ziqian Lu, Pingchao Yang, Chenggang Qin, and Haibo Chen. Characterizing serverless platforms with serverlessbench. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, pages 30–44, 2020. [48] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1187–1204, 2020. [49] Yanqi Zhang, Inigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 724–739, 2021. 246

参考文献按用户规则不翻译。请在原始 PDF 第 15-16 页核对完整参考文献列表。

Quality-Check Note

Required output order: Cover, Confirmed reading questions, Question-answer review, Glossary, Bilingual paper body.

Citations in the QA review use original PDF page numbers and stable block IDs.

Glossary is paper-specific for serverless/FaaS/XFaaS terminology.

Bibliography is preserved but not translated.

Facing pages prioritize logical block alignment rather than exact ACM two-column visual alignment.